

FIȘA 4 - Structuri de date: Liste în Python

Enea Sebastian Paul

Teorie

O listă în Python este o structură de date flexibilă și ordonată, folosită pentru a stoca mai multe elemente într-o singură variabilă. Spre deosebire de alte limbaje de programare, listele în Python pot conține elemente de tipuri diferite în același timp și își pot modifica dimensiunea în mod dinamic în timpul rulării programului.

Elementele unei liste sunt delimitate de paranteze pătrate [] și sunt separate prin virgulă. Fiecare element are o poziție bine definită numită index:

- Indexarea standard începe de la 0 (primul element se află la indexul 0, al doilea la indexul 1 etc.).
- Python permite și indexarea negativă, unde -1 reprezintă ultimul element din listă, -2 reprezintă penultimul element etc.

Funcții:

- `len(lista)` - returnează numărul total de elemente din listă.
- `lista.append(element)` - adaugă un nou element la sfârșitul listei.
- `lista.insert(index, element)` - inserează un element la poziția specificată de index.
- `lista.remove(valoare)` - șterge primul element din listă care are valoarea respectivă.
- `lista.pop(index)` - șterge și returnează elementul de la indexul dat (sau ultimul element dacă indexul lipsește).

Reprezentarea grafică a indexării în liste

Valoare	75	10	23	91	4	38	50
Index Pozitiv	0	1	2	3	4	5	6
Index Negativ	-7	-6	-5	-4	-3	-2	-1

Sintaxă generală

```
nume_lista = [element1, element2, element3]
# Accesare element: nume_lista[index]
# Adăugare element la final: nume_lista.append(valoare)
```

Exemplu de cod 1

```
# Crearea unei liste și manipularea elementelor prin metode de bază
orase = ["Cluj", "București", "Timișoara"]
```

```

print("Lista inițială:", orase)

orase.append("Brașov")    # Adăugăm la final
orase.insert(1, "Iași")   # Inserăm pe poziția 1
print("Lista modificată:", orase)
print("Primul oraș:", orase[0], "| Ultimul oraș:", orase[-1])

```

Exemplu de cod 2

```

# Citirea unei liste de la tastatură și filtrarea elementelor într-o
bucclă for
date = input("Introdu notele separate prin spațiu: ").split()
note = []
for x in date:
    note.append(int(x))

print("Notele introduse sunt:", note)
trecute = 0
for nota in note:
    if nota >= 5:
        trecute += 1
print("Numărul de note de trecere (>= 5) este:", trecute)

```

Exercițiu rezolvat (Model)

Cerință: Scrieți un program care citește o listă de numere întregi de pe o singură linie separate prin spațiu. Programul trebuie să parcurgă lista și să creeze o nouă listă care conține doar numerele pozitive și pare din lista inițială. La final, afișați ambele liste și numărul de elemente din noua listă.

```

date = input("Introdu numerele separate prin spațiu: ").split()
lista_initiala = []
for x in date:
    lista_initiala.append(int(x))

lista_filtrata = []
for nr in lista_initiala:
    if nr > 0 and nr % 2 == 0:
        lista_filtrata.append(nr)

print("Lista inițială:", lista_initiala)
print("Lista filtrată (pare pozitive):", lista_filtrata)
print("Numărul de elemente din lista nouă:", len(lista_filtrata))

```

Exerciții

1. Citește o listă de prețuri (numere reale) de pe o singură linie separate prin spațiu. Scrieți un program care determină și afișează cel mai mic și cel mai mare preț din listă, fără a folosi funcțiile predefinite `min()` și `max()`.

2. Se citește o listă de numere întregi de la tastatură separate prin spațiu. Calculați și afișați media aritmetică a tuturor elementelor din listă care sunt strict mai mari decât valoarea 10.
3. Scrieți un program care de la tastatură citește o listă de numere întregi. Inversați ordinea elementelor din listă fără a folosi funcții speciale de inversare (creați o listă nouă parcurgând lista inițială de la sfârșit spre început) și afișați rezultatul.
4. Se citește o listă de numere întregi separate prin spațiu. Verificați dacă un număr x (citit ulterior de la tastatură) se află în listă. Dacă da, afișați de câte ori apare și la ce index se află prima dată. Dacă nu apare, afișați un mesaj corespunzător.
5. Se citește o listă de note de la tastatură separate prin spațiu. Scrieți un program care majorează cu 1 punct toate notele mai mici decât 10 din listă, iar notele care depășesc valoarea 10 în urma majorării vor rămâne 10. Afișați lista finală modificată.